HAULOS

Senior Engineer Technical Specification
UI Displays + System Design + API Contracts

Prepared for founder review and developer handoff

Primary scope	MVP focus	Phase 2
WA-first heavy vehicle intelligence platform	Routing, fuel, hazards, logbook, live tracking	Escort / traffic warden oversize awareness

1. Product intent

HaulOS is a real-time operating system for heavy transport in Australia. It is not a generic GPS app and it is not only a fleet tracker. The product combines official heavy vehicle access constraints with live driver intelligence to generate safer, legally compliant, road-aware trips.

This specification is written for a senior engineer or technical lead. It defines the UI surfaces, user flows, system architecture, data model, realtime behaviour, API contracts, and core business rules required to build the MVP cleanly and avoid rework.

2. Scope summary

Module	MVP	Phase 2	Notes
Constraint-aware routing	Yes	Expanded	Vehicle profile required before route calculation.
Fuel intelligence	Yes	Expanded	Map markers, confidence score, route alerts.
Hazard reporting	Yes	Expanded	One-tap reporting from driver app.
Rest stop status	Yes	Expanded	Available / Full / Limited where supported.
Digital logbook	Lite	Expanded	Work/rest tracking and fatigue prompts.
Live tracking	Yes	Expanded	Friend / convoy / employer visibility with permissions.

Employer portal	Basic	Expanded	Driver opt-in visibility first; platform later.
Escort / warden mode	No	Yes	Live location broadcast for oversize loads.

3. UI displays and screen intent

The app must feel fast, simple, and usable by a tired driver on the roadside. Dark mode is preferred for practical in-cab use. The driver should be able to complete the key actions with minimal taps and without hunting through menus.

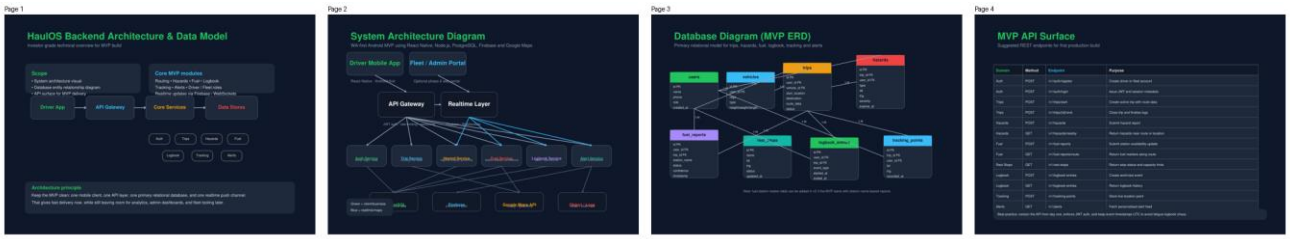

Figure 1. Consolidated HaulOS UI mock-up montage used as the design reference for developer implementation.

3.1 Screen catalogue

Screen	Primary user	Purpose	Critical components	Key actions
Onboarding + vehicle profile	Driver / warden	Capture truck and load constraints before routing	Height, width, length, gross weight, axle/load notes, profile save	Create profile, edit profile, choose active profile
Live map	Driver	Primary operating surface during a trip	Approved route line, hazard markers, fuel markers, rest areas, escort alerts, bottom action bar	Start trip, report, reroute, share trip
Fuel reporting	Driver	Quick report of station status	Buttons for Available, Low Supply, No Diesel, Closed, Restricted; station metadata; timestamp	Submit report, confirm existing report
Hazard reporting	Driver	One-tap live hazard	Hazard type,	Report hazard,

		input	severity, location, optional note/photo later	cancel, update
Dashboard	Driver	Trip overview and shortcuts	Active trip card, alerts, logbook summary, share / tracking controls	Start trip, share trip, view alerts
Logbook (lite)	Driver	Record work/rest and show fatigue status	Clock, work block, rest block, totals, warning banner	Start rest, end rest, add manual correction
Employer visibility	Employer / dispatcher	Monitor only authorised drivers and trips	Driver list, live locations, trip state, risk alerts, compliance summary	View trip, follow driver, acknowledge alert
Escort / warden mode	Traffic warden	Broadcast oversize load movement to nearby drivers	Oversize width, heading, live location, alert radius, status switch	Start escort, stop escort, update width/load details

3.2 UI behaviour requirements

- Vehicle profile must be selected before the first route can be calculated.
- Map must default to the active trip context; the driver should not have to navigate away to see fuel, hazards, and route warnings.
- All report actions should be reachable in two taps or fewer from the live map.
- Status colours must be consistent across map, cards, and alerts: red for blocked/high risk, orange for caution/low supply, green for available/safe, blue for informational/navigation.
- Employer visibility must be permission-based and easy for the driver to revoke.
- Escort mode must clearly identify oversize movement and direction so oncoming drivers can react early.

4. User roles and permissions

Role	Core permissions	Sensitive permissions	Notes
Driver	Create trip, calculate route, report hazards/fuel/rest stop status, log work/rest, share trip	Share live location to employer/friends	Default user role.

Employer / dispatcher	View authorised drivers, active trips, alerts, compliance summaries	No access unless driver or company relationship exists	MVP can be basic web portal or admin view.
Traffic warden / escort	Start escort mode, broadcast location, publish oversize width details	Live location broadcast to nearby drivers	Phase 2 module.
Admin / support	Moderate reports, view audits, manage stations/rest areas	System-wide access	Internal only.

5. Core user flows

5.1 Driver trip start flow

1. Driver opens app and selects an existing vehicle/load profile or creates a new one.
2. App validates required dimensions: height, width, length, weight. Without these values, route calculation is blocked.
3. Driver enters destination and optional trip metadata.
4. Routing engine requests approved route using official network constraints plus live hazards, fuel, and rest area overlays.
5. Map displays recommended route, constraints avoided, fuel risk notices, and active alerts before trip begins.

5.2 Live report flow

6. Driver taps Report on the map.
7. Driver chooses hazard, fuel, or rest stop status.
8. App uses current GPS point or selected station/segment.
9. Report is posted immediately and reflected on the map for nearby relevant users.
10. Subsequent users may confirm or supersede the report, increasing or decreasing confidence.

5.3 Employer monitoring flow

11. Driver explicitly links employer or shares a trip.
12. Employer dashboard shows authorised trips only.
13. Live positions, alerts, and logbook summaries refresh in near real time.
14. Driver can revoke employer visibility at any time unless future enterprise rules require company-managed devices.

5.4 Escort / warden flow (Phase 2)

15. Warden signs in and starts escort mode for a selected oversize movement.
16. Warden enters oversize width and optional load notes.
17. System broadcasts live position, heading, and oversize alert radius.
18. Nearby drivers on impacted segments receive a warning before visual encounter.
19. Warden stops escort mode at trip end; broadcast immediately ceases.

6. Architecture overview

Recommended MVP architecture:

- Mobile app: React Native for Android-first delivery; iOS later if required.
- API layer: Node.js / TypeScript service exposing REST APIs for auth, routing, reports, logbook, tracking, and employer access.
- Operational database: PostgreSQL as the source of truth for users, trips, stations, route segments, reports, and logbook records.
- Realtime delivery: Firebase Cloud Messaging for push notifications plus WebSockets (or Firebase Realtime / Firestore listeners) for active live-map updates.
- Maps and geospatial: Google Maps SDK for display. Heavy vehicle network overlays and restriction data stored internally or refreshed from authoritative WA sources.
- Background workers: report expiry jobs, confidence recalculation, alert fan-out, tracking summarisation, audit tasks.
- Employer portal: lightweight web app consuming the same API layer with scoped role-based access.

7. Data model

Entity	Purpose	Key fields	Relationships
users	Identity and role management	id, role, full_name, phone, email, status, created_at	1:N vehicle_profiles, trips, reports, logbook_entries
vehicle_profiles	Constraint input for routing	id, user_id, name, height_m, width_m, length_m, gross_weight_t, load_type	Belongs to user; referenced by trips and routes
employers	Company account for monitored trips	id, company_name, status	1:N employer_driver_links
employer_driver_links	Permission bridge between driver and employer	id, employer_id, driver_user_id, permissions_json, active	Belongs to employer and driver
trips	Top-level trip record	id, user_id, vehicle_profile_id, origin, destination, status, started_at, ended_at	1:N tracking_points, alerts, trip_reports
route_segments	Normalised network sections used in route calculation	id, segment_code, geometry, source, approved_classes	Referenced by restrictions and trip routes
structure_restrictions	Bridge/powerline/structure	id, route_segment_id,	Belongs to segment

	constraints	type, max_height_m, max_weight_t, notes	
road_restrictions	Segment-level access rules	id, route_segment_id, max_width_m, max_length_m, permit_required, notes	Belongs to segment
fuel_stations	Fuel locations shown in app	id, external_ref, name, lat, lng, operator	1:N fuel_reports
fuel_reports	Live driver fuel updates	id, station_id, user_id, status, confidence_score, created_at, expires_at	Belongs to station and user
hazard_reports	Live hazard reports	id, user_id, trip_id, type, severity, lat, lng, created_at, expires_at	Belongs to user / optional trip
rest_stop_status_reports	Driver-reported rest area status	id, rest_area_id, user_id, status, created_at, expires_at	Belongs to rest area and user
logbook_entries	Work/rest records	id, user_id, entry_type, started_at, ended_at, source	Belongs to user
tracking_points	Compressed live location history	id, trip_id, user_id, lat, lng, heading, speed_kph, recorded_at	Belongs to trip and user
escort_sessions	Phase 2 oversize broadcasts	id, user_id, oversize_width_m, status, started_at, ended_at	1:N escort_tracking_points
escort_tracking_points	Realtime escort positions	id, escort_session_id, lat, lng, heading, speed_kph, recorded_at	Belongs to escort session
alerts	Normalised alert feed	id, user_id, trip_id, alert_type, severity, payload_json, created_at, acknowledged_at	Belongs to user/trip

8. Business rules and engineering rules

- Routing rule: a route cannot be generated unless an active vehicle profile is present.
- Constraint rule: any route that violates a bridge height, powerline height, width, length, or weight restriction must be rejected by the route engine before being returned.
- Fuel confidence rule: confidence starts low on a single report and increases with independent, recent confirmations. Confidence decays over time.
- Report expiry rule: hazard, fuel, and rest stop reports expire automatically after configurable time windows unless reconfirmed.
- Visibility rule: employer monitoring is opt-in at the driver level for the MVP.
- Realtime rule: map updates should appear within seconds for active users in the affected area or trip context.
- Audit rule: all user-generated reports and manual admin changes must be attributable and timestamped.
- Safety rule: UI must favour short actions and high-contrast warnings over dense data entry while the trip is active.

9. API contracts (REST)

All endpoints are versioned under /v1. Authentication is bearer-token based. JSON only for MVP. Exact route names can change, but the functional contract should remain intact.

Domain	Method	Path	Purpose	Auth
Auth	POST	/v1/auth/register	Create account	No
Auth	POST	/v1/auth/login	Login and return access token	No
Vehicle profiles	POST	/v1/vehicle-profiles	Create profile	Yes
Vehicle profiles	GET	/v1/vehicle-profiles	List profiles	Yes
Vehicle profiles	PATCH	/v1/vehicle-profiles/{id}	Update profile	Yes
Trips	POST	/v1/trips	Create/start trip	Yes
Trips	POST	/v1/trips/{id}/end	End trip	Yes
Routing	POST	/v1/routes/calculate	Calculate compliant route	Yes
Fuel	POST	/v1/fuel-reports	Submit station status	Yes
Fuel	POST	/v1/fuel-reports/{id}/confirm	Confirm an existing report	Yes

Fuel	GET	/v1/fuel-stations/nearby	Get nearby stations and live status	Yes
Hazards	POST	/v1/hazard-reports	Submit hazard	Yes
Hazards	GET	/v1/hazard-reports/nearby	Get nearby hazards	Yes
Rest stops	POST	/v1/rest-stop-reports	Submit rest area status	Yes
Rest stops	GET	/v1/rest-areas/nearby	Get nearby rest areas and live status	Yes
Logbook	POST	/v1/logbook-entries	Create work/rest entry	Yes
Logbook	GET	/v1/logbook-entries	List user logbook entries	Yes
Tracking	POST	/v1/tracking-points	Ingest live location ping	Yes
Employers	POST	/v1/employers/link	Link driver to employer	Yes
Employers	GET	/v1/employers/{id}/live	Employer dashboard live feed	Yes
Escort	POST	/v1/escort-sessions	Start escort session	Yes
Escort	POST	/v1/escort-sessions/{id}/stop	Stop escort session	Yes
Escort	POST	/v1/escort-tracking-points	Ingest escort location	Yes

9.1 Example request and response contracts

POST /v1/routes/calculate

```
Request
{
  "vehicle_profile_id": "vp_123",
  "origin": {"lat": -31.95, "lng": 115.86},
  "destination": {"lat": -32.04, "lng": 115.75},
  "trip_context": {"share_with_employer": true}
}

Response
{
```

```
  "route_id": "rt_987",
  "status": "ok",
  "distance_km": 42.3,
  "duration_min": 58,
  "warnings": [
    "Low clearance structure avoided",
    "Fuel low-supply zone within 25 km of destination"
  ],
  "segments": [...],
  "fuel_notices": [...],
  "hazard_notices": [...]
}
```

POST /v1/fuel-reports

Request

```
{
  "station_id": "stn_456",
  "status": "NO_DIESEL",
  "note": "No diesel on pump 2 and 3"
}
```

Response

```
{
  "report_id": "fr_999",
  "status": "accepted",
  "confidence_score": 0.34,
  "expires_at": "2026-03-19T14:00:00Z"
}
```

POST /v1/hazard-reports

Request

```
{
  "type": "DEBRIS",
  "severity": "HIGH",
  "lat": -30.12,
  "lng": 116.02,
  "note": "Tyre fragments in left lane"
}
```

Response

```
{
  "report_id": "hz_111",
  "status": "accepted",
  "broadcasted": true
}
```

POST /v1/escort-sessions

Request

```
{
  "oversize_width_m": 8.0,
  "load_note": "Oversize movement northbound",
  "alert_radius_km": 10
}
```

Response

```
{
  "escort_session_id": "esc_444",
  "status": "active"
}
```

10. Realtime event model

For active map sessions, WebSocket or equivalent push events should be used rather than repeated polling. Suggested event topics:

- trip.started
- route.recalculated
- fuel.report.created
- fuel.report.confirmed
- hazard.report.created
- rest_stop.report.created
- alert.created
- tracking.updated
- escort.started
- escort.updated
- escort.stopped

11. Non-functional requirements

- Security: authenticated APIs, role-based access control, secure token storage on device, auditable admin actions.
- Performance: route calculation should feel responsive; driver report posting should complete within a few seconds on normal mobile data.
- Reliability: stale report cleanup and confidence recalculation must run even if no user is currently online.
- Observability: structured logs, request IDs, error tracking, and metrics for route time, alert fan-out, report rates, and app crashes.
- Offline tolerance: if connectivity drops, queue non-critical report submissions and retry when signal returns; clearly indicate unsent state.
- Privacy: only authorised parties should see live location; employer visibility must be revocable for MVP.
- Configurability: escort thresholds, report expiry windows, and confidence rules should be configurable rather than hardcoded.

12. Developer handoff deliverables

- Mobile app codebase in React Native with environment configuration and build instructions.
- Node.js / TypeScript backend with tested REST endpoints and migrations.
- PostgreSQL schema migrations and seed data for stations, network segments, and example restrictions.
- Basic employer-facing web dashboard or admin view for authorised trip visibility.
- Realtime layer for map updates and push notification wiring.
- Documentation covering local setup, deployment steps, environment variables, API auth, and operational runbook.
- Repository access, weekly progress cadence, and milestone review demos.

13. Acceptance criteria

- A driver can create a vehicle profile and the app refuses route calculation until required dimensions are present.
- A calculated route avoids known restriction violations for the active profile.
- A driver can post a fuel, hazard, or rest stop update in two taps or fewer from the map context.
- Nearby users receive relevant updates within a reasonable realtime window.
- A driver can link an employer and the employer can view only authorised trips and drivers.
- Logbook entries can be recorded, listed, and summarised.
- Phase 2 architecture can be extended to escort mode without replacing core identity, trips, or tracking foundations.